МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

Лабораторна робота №4

з курсу: «Технології побудови розподілених баз даних та знань»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2026

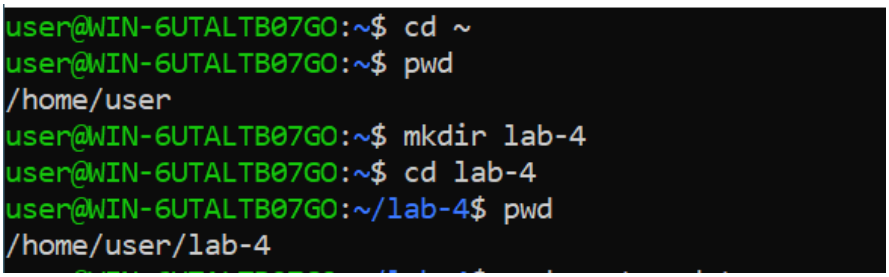
Практичні завдання

1) Виконуємо налаштування оточення кластера.
2) З тестовими даним, відмінними від вказаних, виконуємо ознайомчу частину.
3) Зчитаємо кількість документів по “tablets”. Оновимо екземпляр даних. Зчитаємо ідентичний документ з вузла, котрий реплікує (дублює) вміст основного вузла.
4) Утилітою docker-compose й опцією up включимо додатково два керівних вузла й два вузла даних. Дозапишемо 5 документів. З’єднаємо таблиці з декількох шардів.
5) Утилітою docker-compose й опцією down вимкнемо шард і опцією up включимо новий. В UI зчитаємо вміст кластера.

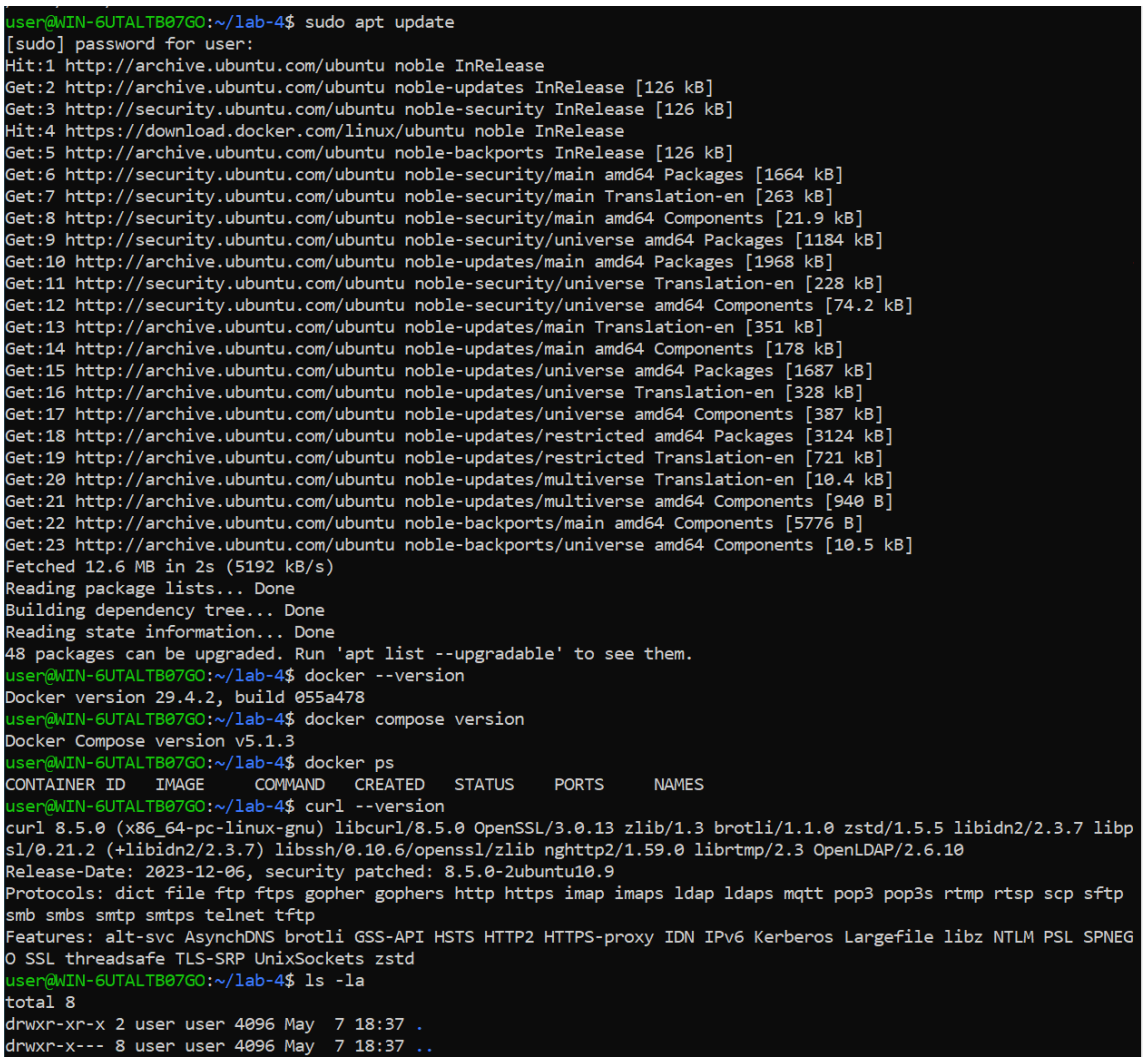
Хід роботи

1. Підготовка та налаштування серидовища

У межах першого етапу лабораторної роботи було виконано налаштування робочого середовища WSL Ubuntu Linux для подальшого розгортання YugabyteDB-кластера. Спочатку було здійснено перехід у домашню директорію користувача та створено окремий каталог lab-4, у якому надалі зберігатиметься конфігураційний файл docker-compose.yml та інші матеріали лабораторної роботи.



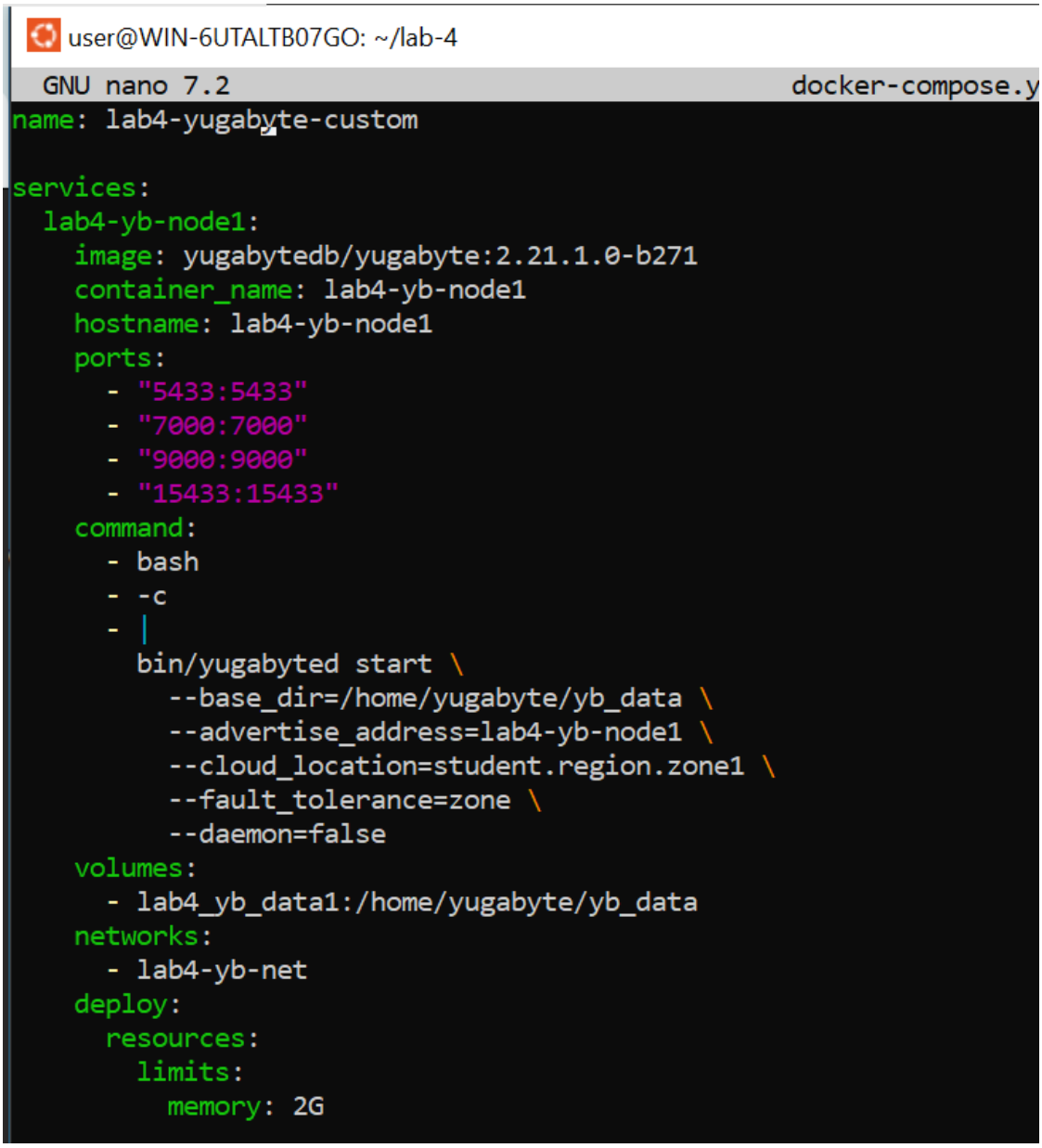
Після цього було виконано оновлення списку пакетів Ubuntu за допомогою команди sudo apt update. Це дозволило отримати актуальну інформацію про доступні пакети з репозиторіїв системи. Далі було перевірено наявність основних інструментів, необхідних для виконання роботи: Docker, Docker Compose та curl. Docker використовується для запуску контейнерів, Docker Compose дає змогу описати та запустити багатовузловий кластер за допомогою одного YAML-файлу, а curl може використовуватися для перевірки доступності веб-інтерфейсів або HTTP-сервісів.



Результати перевірки показали, що Docker версії 29.4.2, Docker Compose версії v5.1.3 та curl версії 8.5.0 встановлені й доступні в системі. Команда docker ps виконалася без помилок доступу, що підтверджує коректну роботу Docker daemon. Оскільки контейнери ще не запускалися, список активних контейнерів був порожнім. Таким чином, середовище готове до наступного етапу, а саме створення та налаштування файлу docker-compose.yml для запуску YugabyteDB-кластера.

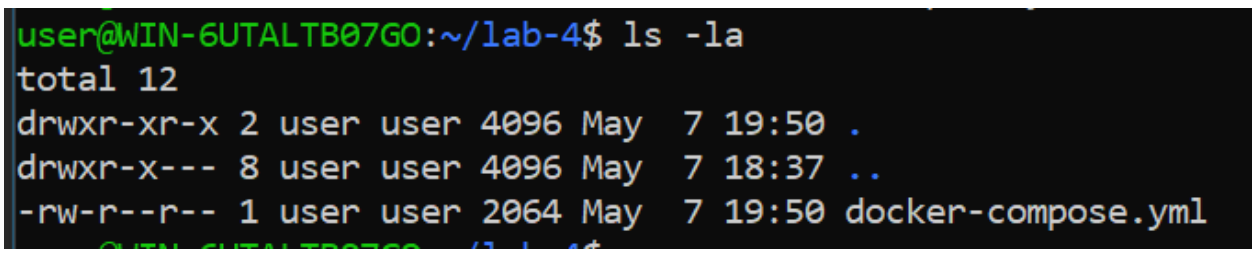
2. Підготовка yml файлу

Перед створенням конфігураційного файлу було перевірено поточну робочу директорію. Це потрібно для того, щоб файл docker-compose.yml був створений саме в каталозі лабораторної роботи lab-4. Docker Compose за замовчуванням шукає файл з назвою docker-compose.yml у поточній директорії, тому важливо виконувати команди саме з цієї папки.



У цьому файлі описано конфігурацію YugabyteDB-кластера, який складається з трьох вузлів: lab4-yb-node1, lab4-yb-node2 і lab4-yb-node3. Перший вузол запускається як початковий вузол кластера, а другий і третій підключаються до нього за допомогою параметра --join=lab4-yb-node1.

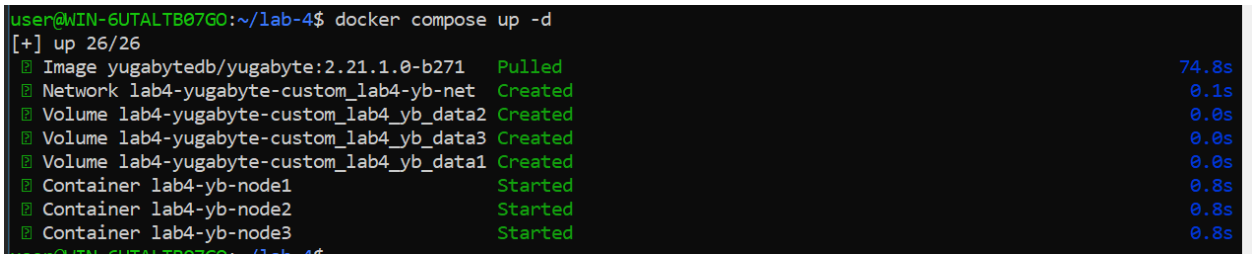
Конфігурація відрізняється від прикладу з методичних матеріалів, оскільки використано власні назви контейнерів, Docker-мережі, volumes і зон розташування. Це дозволяє виконати вимогу лабораторної роботи щодо використання власного варіанту налаштування, але при цьому зберігається правильна логіка розгортання розподіленого YugabyteDB-кластера.

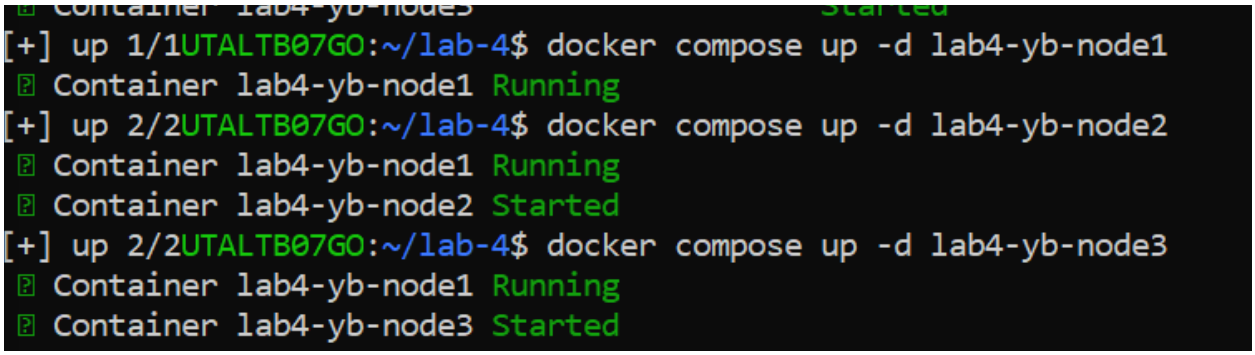


3. Виконуємо налаштування оточення кластера

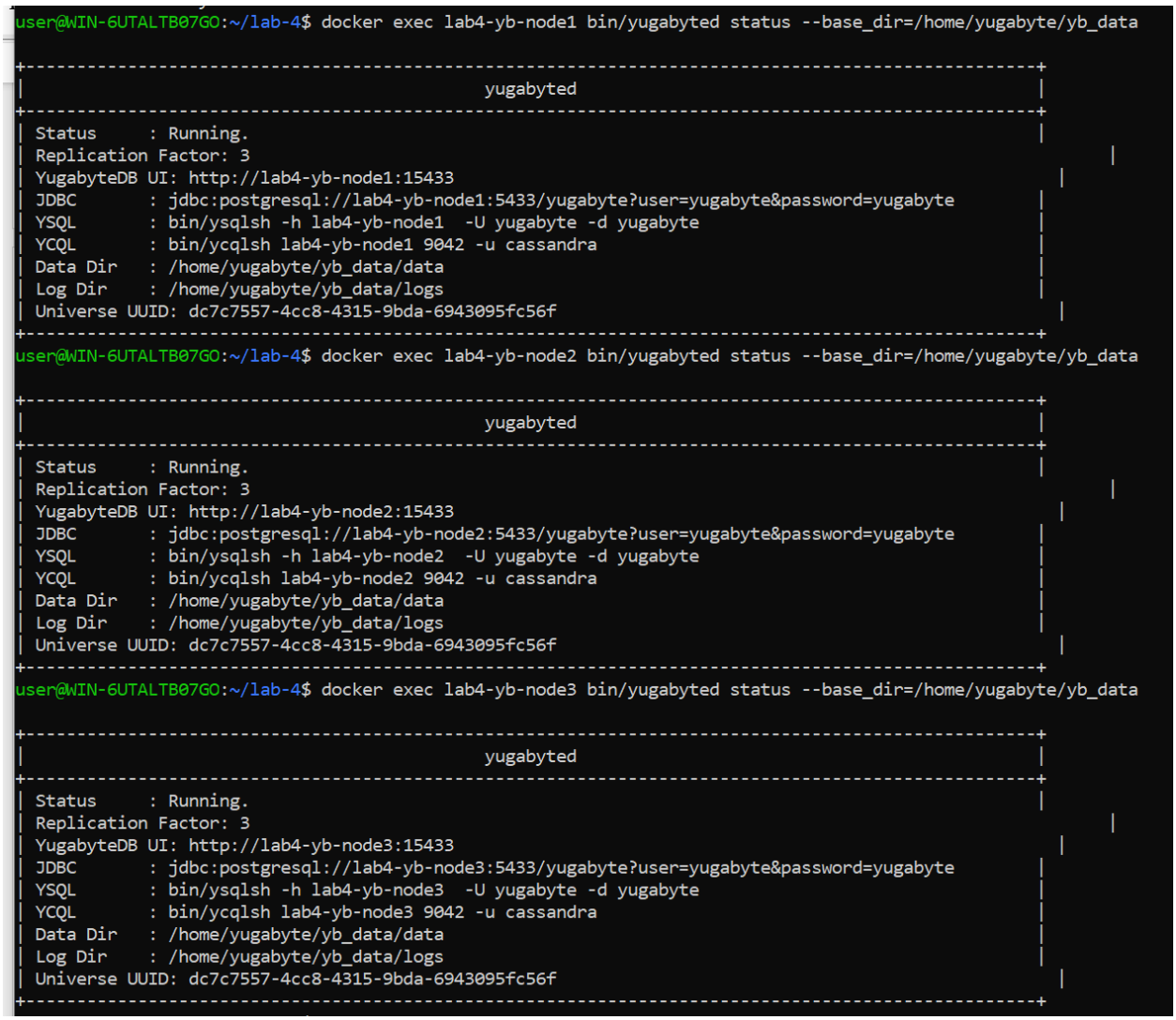
У цьому завданні ми запускаємо YugabyteDB-кластер, який уже описали у файлі docker-compose.yml. Тобто пункт 1 з методички практично складається з запуску середовища, перевірки контейнерів, перевірки статусу вузлів і перевірки доступу до веб-інтерфейсу та SQL-консолі.

Команда docker compose up -d запускає всі сервіси, описані у файлі docker-compose.yml, у фоновому режимі. У цьому випадку запускаються три контейнери YugabyteDB, які утворюють розподілений кластер. Параметр -d означає detached mode, тобто контейнери продовжують працювати у фоні, а термінал залишається доступним для подальших команд. Під час запуску Docker також створює окрему мережу для взаємодії вузлів і volumes для збереження даних кожного вузла.





У конфігурації було описано три вузли YugabyteDB: lab4-yb-node1, lab4-yb-node2 і lab4-yb-node3. Для кожного вузла було встановлено обмеження пам’яті 1GB, що дозволило стабільно запустити кластер у середовищі WSL. Перший вузол виступив початковим вузлом кластера, а другий і третій вузли підключилися до нього за допомогою параметра --join=lab4-yb-node1.



Після запуску контейнерів було виконано перевірку стану кожного вузла за допомогою команди yugabyted status. Усі три вузли мали статус Running, коефіцієнт реплікації Replication Factor: 3, а також однаковий Universe UUID. Це підтверджує, що вузли успішно об’єдналися в один YugabyteDB-кластер і готові до подальшої роботи з розподіленими таблицями, запитами та реплікацією.

4. З тестовими даними, відмінними від вказаних, виконуємо ознайомчу частину

У цьому пункті ми створимо власну базу даних, власні таблиці та додамо власні тестові записи. Це важливо, бо в завданні прямо вказано виконувати ознайомчу частину з тестовими даними, відмінними від наведених у методичці. Кластер уже запущений коректно: у твоєму логі всі три вузли мають статус Running, Replication Factor: 3 і однаковий Universe UUID, тобто вони належать до одного YugabyteDB-кластера**.**

Для роботи з YugabyteDB використовується консольна утиліта ysqlsh, яка є SQL-клієнтом для YSQL. Інтерфейс YSQL сумісний із PostgreSQL, тому дозволяє виконувати звичайні SQL-команди: створення баз даних, таблиць, вставку записів і виконання запитів. Підключення виконується до першого вузла lab4-yb-node1, але запити обробляються в межах усього кластера.

```
user@WIN-6UTALTB07GO:~/lab-4$ docker exec -it lab4-yb-node1 bin/ysqlsh -h lab4-yb-node1 -p 5433 -U yugabyte
ysqlsh (11.2-YB-2.21.1.0-b0)
Type "help" for help.

yugabyte=#
```

Створимо таблиці customers, orders, activity_logs, та перевіримо їх наявність :

```
lab4_shop=# CREATE TABLE customers (
```

```
lab4_shop=# CREATE TABLE orders
```

```
lab4_shop=# CREATE TABLE activity_logs
```

```
lab4_shop=# \dt
             List of relations
 Schema |     Name      | Type  |  Owner
--------+---------------+-------+----------
 public | activity_logs | table | yugabyte
 public | customers     | table | yugabyte
 public | orders        | table | yugabyte
(3 rows)

lab4_shop=#
```

Далі виконаємо додавання тестових даних

Додаємо клієнтів

```
lab4_shop=# INSERT INTO customers
```

Додаємо замовлення

```
lab4_shop=# INSERT INTO orders
```

Додаємо JSONB-події

```
lab4_shop=# INSERT INTO activity_logs
```

Виконаємо перевірочні запити

Перевірити клієнтів

```
lab4_shop=# SELECT id, email, full_name, city FROM customers;
  id  |           email           |   full_name   |  city
------+---------------------------+---------------+--------
 c005 | ivan.petrenko@example.com | Ivan Petrenko | Dnipro
 c002 | olena.koval@example.com   | Olena Koval   | Lviv
 c004 | maria.bondar@example.com  | Maria Bondar  | Kharkiv
 c001 | nikita.sydko@example.com  | Nikita Sydko  | Kyiv
 c003 | andrii.melnyk@example.com | Andrii Melnyk | Odesa
(5 rows)
```

JOIN-запит

```
lab4_shop=# SELECT c.full_name, c.city, o.product_name, o.amount, o.orde
N c.id = o.customer_id ORDER BY o.amount DESC;
   full_name   |  city   |    product_name     | amount  | order_status
---------------+---------+---------------------+---------+--------------
 Olena Koval   | Lviv    | Mechanical Keyboard | 2499.00 | paid
 Ivan Petrenko | Dnipro  | Web Camera          | 1599.00 | new
 Andrii Melnyk | Odesa   | USB-C Hub           | 1199.00 | processing
 Maria Bondar  | Kharkiv | Laptop Stand        |  999.00 | paid
 Nikita Sydko  | Kyiv    | Wireless Mouse      |  799.00 | paid
(5 rows)
```

Запит до JSONB

```
lab4_shop=# SELECT event_type, payload ->> 'source' AS source,
Y event_type, payload ->> 'source' ORDER BY event_count DESC;
 event_type  | source | event_count
-------------+--------+-------------
 add_to_cart | mobile |           1
 payment     | web    |           1
 login       | web    |           1
(3 rows)
```

Кількість записів у таблицях

```
lab4_shop=# SELECT 'customers' AS table_name, COUNT(*)
table_name, COUNT(*) AS rows_count FROM orders UNION AL
ount FROM activity_logs;
  table_name   | rows_count
---------------+------------
 customers     |          5
 orders        |          5
 activity_logs |          3
(3 rows)
```

5. Зчитаємо кількість документів по “tablets”. Оновимо екземпляр даних. Зчитаємо ідентичний документ з вузла, котрий реплікує (дублює) вміст основного вузла.

Зчитаємо кількість документів по “tablets”.

Створюємо таблицю з поділом на tablets

```
lab4_shop=# CREATE TABLE tablet_documents (doc_id INT PRIMARY KEY, title VARCHAR(100) NOT NULL, category VARCHAR(
50) NOT NULL, doc_value INT NOT NULL, updated_at TIMESTAMPTZ DEFAULT NOW()) SPLIT INTO 3 TABLETS;
CREATE TABLE
```

Додаємо документи

```
lab4_shop=# INSERT INTO tablet_documents (doc_id, title, category, doc_value) VALUES (1, 'Sensor report Kyiv', 's
ensor', 10), (2, 'Payment event Lviv', 'payment', 20), (3, 'User activity Odesa', 'activity', 30), (4, 'Order doc
ument Dnipro', 'order', 40), (5, 'System log Kharkiv', 'log', 50), (6, 'Delivery status Kyiv', 'delivery', 60), (
7, 'Product view Lviv', 'activity', 70), (8, 'Cart update Odesa', 'cart', 80), (9, 'Payment retry Dnipro', 'payme
nt', 90), (10, 'Support ticket Kharkiv', 'support', 100), (11, 'Warehouse update Kyiv', 'warehouse', 110), (12, '
Client feedback Lviv', 'feedback', 120);
INSERT 0 12
```

Зчитуємо кількість документів по tablets

```
lab4_shop=# SELECT CASE WHEN yb_hash_code(doc_id) < 21845 THEN 'tablet_1' WHEN yb_hash_code(doc_id) < 43690 THEN
'tablet_2' ELSE 'tablet_3' END AS tablet_range, COUNT(*) AS documents_count FROM tablet_documents GROUP BY tablet
_range ORDER BY tablet_range;
 tablet_range | documents_count
--------------+-----------------
 tablet_1     |               6
 tablet_2     |               3
 tablet_3     |               3
(3 rows)
```

Оновимо екземпляр даних

```
lab4_shop=# UPDATE tablet_documents SET title = 'UPDATED Payment event Lviv', doc_value = 777, updated_at = NOW()
 WHERE doc_id = 2;
UPDATE 1
lab4_shop=# SELECT doc_id, title, category, doc_value, updated_at FROM tablet_documents WHERE doc_id = 2;
 doc_id |           title            | category | doc_value |          updated_at
--------+----------------------------+----------+-----------+-------------------------------
      2 | UPDATED Payment event Lviv | payment  |       777 | 2026-05-07 18:18:19.588569+00
(1 row)
```

Зчитаємо ідентичний документ з вузла, котрий реплікує (дублює) вміст основного вузла.

Вийдемо з консолі, та переглянемо ідентичні документи з реплік (node 2 node 3)

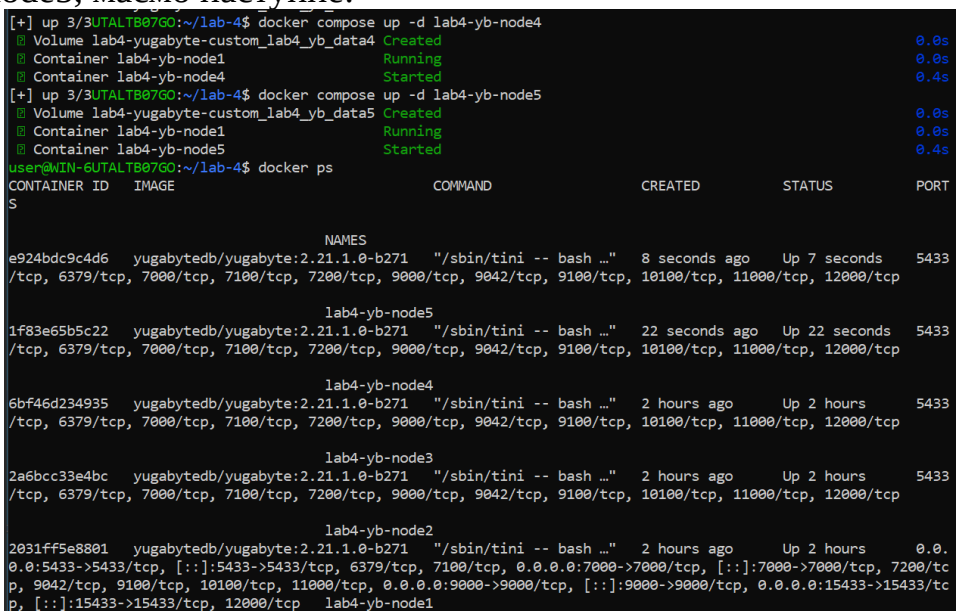
```
lab4_shop=# \q
user@WIN-6UTALTB07GO:~/lab-4$ docker exec -it lab4-yb-node2 bin/ysqlsh -h lab4-yb-node2 -p 5433 -U yugabyte -d la
b4_shop -c "SELECT doc_id, title, category, doc_value, updated_at FROM tablet_documents WHERE doc_id = 2;"
 doc_id |           title            | category | doc_value |          updated_at
--------+----------------------------+----------+-----------+-------------------------------
      2 | UPDATED Payment event Lviv | payment  |       777 | 2026-05-07 18:18:19.588569+00
(1 row)

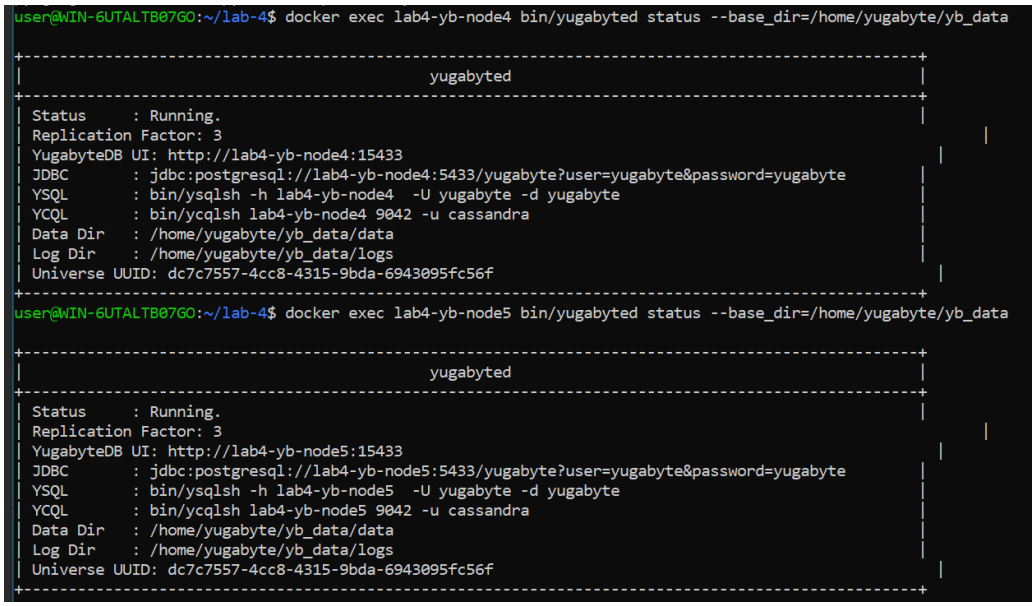
user@WIN-6UTALTB07GO:~/lab-4$ docker exec -it lab4-yb-node3 bin/ysqlsh -h lab4-yb-node3 -p 5433 -U yugabyte -d la
b4_shop -c "SELECT doc_id, title, category, doc_value, updated_at FROM tablet_documents WHERE doc_id = 2;"
 doc_id |           title            | category | doc_value |          updated_at
--------+----------------------------+----------+-----------+-------------------------------
      2 | UPDATED Payment event Lviv | payment  |       777 | 2026-05-07 18:18:19.588569+00
(1 row)
```

Тобто, по суті було створено таблицю tablet_documents з поділом на 3 tablets. Після додавання 12 документів було виконано підрахунок кількості записів за умовними діапазонами tablets через функцію yb_hash_code (). Далі один документ із doc_id = 2 було оновлено на основному вузлі кластера. Після цього той самий документ було прочитано через другий і третій вузли. Однаковий результат підтверджує, що дані доступні через різні вузли YugabyteDB-кластера, а реплікація забезпечує узгоджений стан записів.

6. Утилітою docker-compose й опцією up включимо додатково два керівних вузла. Дозапишемо 5 документів. З'єднаємо таблиці з декількох шардів.

Оновивши версію файлу docker-compose.yml, додавши туди lab4-yb-node4 і lab4-yb-node5, маємо наступне:





Підключитися до бази lab4_shop

```
user@WIN-6UTALTB07GO:~/lab-4$ docker exec -it lab4-yb-node1 bin/ysqlsh -h lab4-yb-node1 -p 5433 -U yugabyte -d la
b4_shop
ysqlsh (11.2-YB-2.21.1.0-b0)
Type "help" for help.
```

Дозапис 5 документів

```
lab4_shop=# INSERT INTO tablet_documents (doc_id, title, category, doc_value) VALUES (13, 'Extra analytics report
 Kyiv', 'analytics', 130), (14, 'Extra payment audit Lviv', 'payment', 140), (15, 'Extra user session Odesa', 'ac
tivity', 150), (16, 'Extra delivery route Dnipro', 'delivery', 160), (17, 'Extra support message Kharkiv', 'suppo
rt', 170);
INSERT 0 5
```

Та перевірка загальної кількості документів

```
lab4_shop=# SELECT COUNT(*) AS total_documents FROM tablet_documents;
 total_documents
-----------------
              17
(1 row)
```

Перевірка розподілу документів по tablets

```
lab4_shop=# SELECT CASE WHEN yb_hash_code(doc_id) < 21845 THEN 'tablet_1' WHEN yb_hash_code(doc_id) < 43690 THEN
'tablet_2' ELSE 'tablet_3' END AS tablet_range, COUNT(*) AS documents_count FROM tablet_documents GROUP BY tablet
_range ORDER BY tablet_range;
 tablet_range | documents_count
--------------+-----------------
 tablet_1     |               9
 tablet_2     |               4
 tablet_3     |               4
(3 rows)
```

З’єднати таблиці з декількох шардів

```
lab4_shop=# SELECT c.full_name, c.city, o.product_name, o.amount, d.title AS document_title, d.category FROM cust
omers c JOIN orders o ON c.id = o.customer_id JOIN tablet_documents d ON d.category = o.order_status OR d.categor
y = 'payment' ORDER BY c.full_name, d.doc_id LIMIT 10;
   full_name   |  city   |  product_name  | amount |       document_title       | category
---------------+---------+----------------+--------+----------------------------+----------
 Andrii Melnyk | Odesa   | USB-C Hub      | 1199.00 | UPDATED Payment event Lviv | payment
 Andrii Melnyk | Odesa   | USB-C Hub      | 1199.00 | Payment retry Dnipro       | payment
 Andrii Melnyk | Odesa   | USB-C Hub      | 1199.00 | Extra payment audit Lviv   | payment
 Ivan Petrenko | Dnipro  | Web Camera     | 1599.00 | UPDATED Payment event Lviv | payment
 Ivan Petrenko | Dnipro  | Web Camera     | 1599.00 | Payment retry Dnipro       | payment
 Ivan Petrenko | Dnipro  | Web Camera     | 1599.00 | Extra payment audit Lviv   | payment
 Maria Bondar  | Kharkiv | Laptop Stand   |  999.00 | UPDATED Payment event Lviv | payment
 Maria Bondar  | Kharkiv | Laptop Stand   |  999.00 | Payment retry Dnipro       | payment
 Maria Bondar  | Kharkiv | Laptop Stand   |  999.00 | Extra payment audit Lviv   | payment
 Nikita Sydko  | Kyiv    | Wireless Mouse |  799.00 | UPDATED Payment event Lviv | payment
(10 rows)
```

Було виконано запит з об’єднанням таблиць customers, orders і tablet_documents. Ці таблиці зберігаються в розподіленому кластері та можуть бути поділені на різні tablets. Для користувача це виглядає як звичайний SQL JOIN, однак на рівні YugabyteDB дані можуть фізично розміщуватися на різних вузлах і shards/tablets. Це демонструє можливість виконання запитів до розподілених даних.

У результаті виконання практичного завдання кластер YugabyteDB було розширено за допомогою Docker Compose. До трьох наявних вузлів було додано ще два вузли lab4-yb-node4 і lab4-yb-node5, які підключилися до існуючого кластера через параметр --join=lab4-yb-node1. Після цього до таблиці tablet_documents було дозаписано 5 нових документів, а загальна кількість записів збільшилася до 17. Далі було перевірено розподіл документів по tablets і виконано JOIN-запит між кількома таблицями. Це підтверджує, що YugabyteDB дозволяє працювати з розподіленими даними через звичайні SQL-запити.

7) Утилітою docker-compose й опцією down вимкнемо шард і опцією up включимо новий. В UI зчитаємо вміст кластера.

Стан кластера ДО вимкнення

Universe Summary

Cluster UUID	Total Live TServers	Total Blacklisted TServers	Total Dead TServers	User Tablet-Peers	System Tablet-Peers	Hidden Tablet-Peers	Active Tablet-Peers	Tablet Peer Limit
Primary Cluster d66e4159-dab6-4bbe-b272-64e1a66fa9af	5	0	0	39	27	0	66	N/A

Tablet Servers

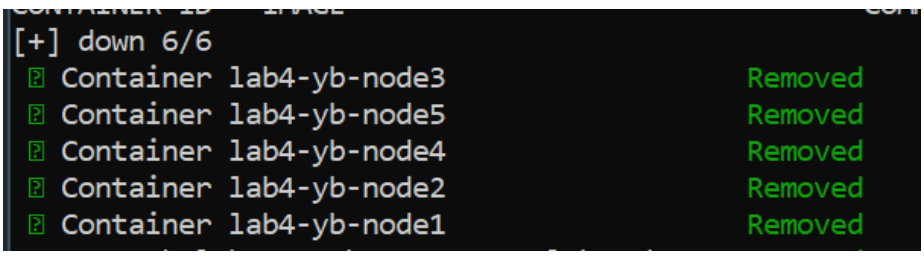
Primary Cluster UUID: d66e4159-dab6-4bbe-b272-64e1a66fa9af

Server	Time since heartbeat	Status & Uptime	User Tablet-Peers / Leaders	RAM Used	Num SST Files	Total SST Files Size	Uncompressed SST Files Size	Read ops/sec	Write ops/sec	Cloud	Region	Zone	System Tablet-Peers / Leaders	Active Tablet-Peers
lab4-yb-node1:9000 31b473a1ad5b4b119fbac20d92a612bd	0.1s	ALIVE: 2:09:56	13 / 4	83.21 MB	1	73.42 KB	161.42 KB	0	0.8	student	region	zone1	9 / 4	22
lab4-yb-node2:9000 e365eba1216f4086906424f514d80323	0.1s	ALIVE: 2:09:46	13 / 5	65.21 MB	1	73.42 KB	161.42 KB	0	0	student	region	zone2	9 / 2	22
lab4-yb-node3:9000 9756a35971d64a1bb65259b9b231d0de	0.1s	ALIVE: 2:09:31	13 / 4	64.94 MB	1	73.42 KB	161.42 KB	0	0	student	region	zone3	9 / 3	22
lab4-yb-node4:9000 c42fa4c6eeac4a0288623fc1e3b97bec	0.4s	ALIVE: 0:15:06	0 / 0	23.40 MB	0	0 B	0 B	0	0	student	region	zone4	0 / 0	0
lab4-yb-node5:9000 7698f90addf94a6a84c871e2b1dac88b	0.3s	ALIVE: 0:14:50	0 / 0	23.18 MB	0	0 B	0 B	0	0	student	region	zone5	0 / 0	0

User tables

Keyspace	Table Name	State	Message	UUID	YSQL OID	Hidden	On-disk size
lab4_shop	customers	Running	Current schema version=2	00004000000030008000000000400c	16396	false	• Total: 3.00M • WAL Files: 3.00M • SST Files: 0B • SST Files Uncompressed: 0B
lab4_shop	orders	Running	Current schema version=1	00004000000030008000000000404014	16404	false	• Total: 3.00M • WAL Files: 3.00M • SST Files: 0B • SST Files Uncompressed: 0B
lab4_shop	activity_logs	Running		00004000000030008000000000401f	16415	false	• Total: 1.00M • WAL Files: 1.00M • SST Files: 0B • SST Files Uncompressed: 0B
lab4_shop	tablet_documents	Running		00004000000030008000000000404025	16421	false	• Total: 3.00M • WAL Files: 3.00M • SST Files: 0B • SST Files Uncompressed: 0B

Вимкнути шард через docker compose down



Увімкнути новий склад кластера через up

```
user@WIN-6UTALTB07GO:~/lab-4$ docker compose up -d lab4-yb-node1
[+] up 2/2
 ✔ Network lab4-yugabyte-custom_lab4-yb-net Created

0.0s ✔ Container lab4-yb-node1                Started

user@WIN-6UTALTB07GO:~/lab-4$ docker compose up -d lab4-yb-node2 lab4-yb-node3 lab4-yb-node4
[+] up 4/4
 ✔ Container lab4-yb-node1 Running                                                    0.0s
 ✔ Container lab4-yb-node2 Started                                                    0.6s
 ✔ Container lab4-yb-node3 Started                                                    0.6s
 ✔ Container lab4-yb-node4 Started                                                    0.5s
user@WIN-6UTALTB07GO:~/lab-4$ docker compose up -d lab4-yb-node6
[+] up 3/3
 ✔ Volume lab4-yugabyte-custom_lab4_yb_data6 Created                                  0.0s
 ✔ Container lab4-yb-node1                   Running                                  0.0s
 ✔ Container lab4-yb-node6                   Started                                  0.4s
```

Перевірка статусу нового вузла

```
user@WIN-6UTALTB07GO:~/lab-4$ docker exec lab4-yb-node6 bin/yugabyted status --base_dir=/home/yugabyte/yb_data

+-------------------------------------------------------------------------------------------------+
|                                          yugabyted                                              |
+-------------------------------------------------------------------------------------------------+
| Status     : Running.                                                                           |
| Replication Factor: 3                                                                             |
| YugabyteDB UI: http://lab4-yb-node6:15433                                                       |
| JDBC       : jdbc:postgresql://lab4-yb-node6:5433/yugabyte?user=yugabyte&password=yugabyte      |
| YSQL       : bin/ysqlsh -h lab4-yb-node6  -U yugabyte -d yugabyte                               |
| YCQL       : bin/ycqlsh lab4-yb-node6 9042 -u cassandra                                         |
| Data Dir   : /home/yugabyte/yb_data/data                                                        |
| Log Dir    : /home/yugabyte/yb_data/logs                                                        |
| Universe UUID: dc7c7557-4cc8-4315-9bda-6943095fc56f                                           |
+-------------------------------------------------------------------------------------------------+
```

Перевірка що дані збереглися

```
+-------------------------------------------------------------------------------------------------+
user@WIN-6UTALTB07GO:~/lab-4$ docker exec -it lab4-yb-node1 bin/ysqlsh -h lab4-yb-node1 -p 5433 -U yugabyte -d la
b4_shop -c "SELECT COUNT(*) AS total_documents FROM tablet_documents;"
 total_documents
-----------------
              17
(1 row)
```

Перевірка кластеру у Master UI після заміни шарду

Tablet Servers

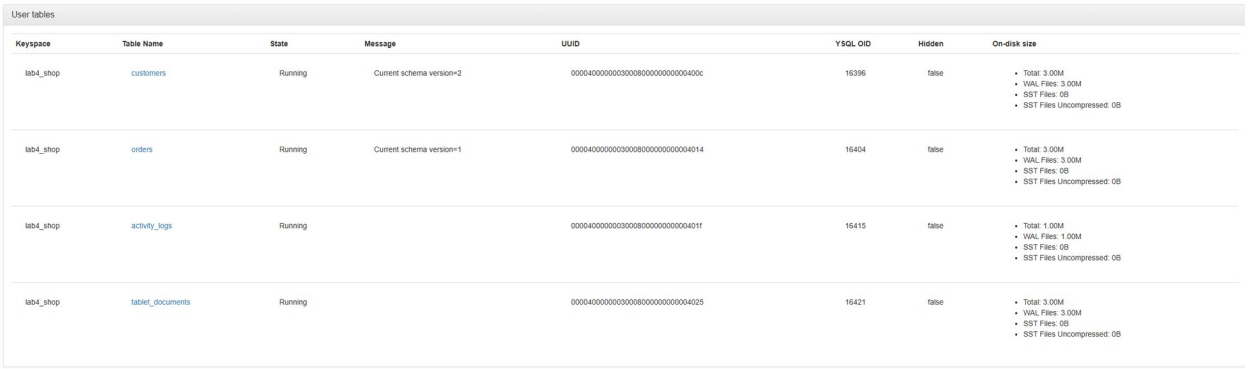
Primary Cluster UUID: d66e4159-dab6-4bbe-b272-64e1a66fa9af

Server	Time since heartbeat	Status & Uptime	User Tablet-Peers / Leaders	RAM Used	Num SST Files	Total SST Files Size	Uncompressed SST Files Size	Read ops/sec	Write ops/sec	Cloud	Region	Zone	System Tablet-Peers / Leaders	Active Tablet-Peers
lab4-yb-node1:9000 31b473a1ad5b4b119fbac20d92a612bd	0.9s	ALIVE: 0:04:02	13 / 4	70.07 MB	1	73.42 KB	161.42 KB	0	0	student	region	zone1	9 / 3	22
lab4-yb-node2:9000 e365eba1216f40869064 24f514d80323	0.9s	ALIVE: 0:04:07	13 / 5	66.49 MB	1	73.42 KB	161.42 KB	0	0	student	region	zone2	9 / 2	22
lab4-yb-node3:9000 9756a35971d64a1bb66259b9b231d0de	0.9s	ALIVE: 0:04:05	13 / 4	72.93 MB	1	73.42 KB	161.42 KB	0	0	student	region	zone3	9 / 4	22
lab4-yb-node4:9000 c42fa4c6eeac4a0288623fc1e3b97bec	0.4s	ALIVE: 0:04:04	0 / 0	20.46 MB	0	0 B	0 B	0	0	student	region	zone4	0 / 0	0
lab4-yb-node6:9000 9eab8a94e21a4c08b2445b7225a9adc4	0.0s	ALIVE: 0:04:02	0 / 0	24.06 MB	0	0 B	0 B	0	0	student	region	zone6	0 / 0	0

На цьому скріні видно, що після заміни шарду активні такі tablet-сервери:

lab4-yb-node1 lab4-yb-node2 lab4-yb-node3 lab4-yb-node4 lab4-yb-node6

А lab4-yb-node5 серед активних уже немає.



User tables

Keyspace	Table Name	State	Message	UUID	YSQL OID	Hidden	On-disk size
lab4_shop	customers	Running	Current schema version=2	00004000000030008000000000400c	16396	false	• Total: 3.00M • WAL Files: 3.00M • SST Files: 0B • SST Files Uncompressed: 0B
lab4_shop	orders	Running	Current schema version=1	00004000000030008000000000004014	16404	false	• Total: 3.00M • WAL Files: 3.00M • SST Files: 0B • SST Files Uncompressed: 0B
lab4_shop	activity_logs	Running		00004000000030008000000000401f	16415	false	• Total: 1.00M • WAL Files: 1.00M • SST Files: 0B • SST Files Uncompressed: 0B
lab4_shop	tablet_documents	Running		00004000000030008000000000004025	16421	false	• Total: 3.00M • WAL Files: 3.00M • SST Files: 0B • SST Files Uncompressed: 0B

після вимкнення старого шарду і запуску нового складу кластера в Master UI залишилися таблиці бази lab4_shop:

1. customers
2. orders
3. activity_logs
4. tablet_documents

Командою docker compose down було вимкнено попередній склад кластера зі старим шардом lab4-yb-node5. Після цього за допомогою docker compose up було запущено новий склад кластера, у якому замість lab4-yb-node5 працює новий вузол lab4-yb-node6. Перевірка docker ps показала п’ять активних контейнерів, а команда yugabyted status підтвердила, що новий вузол має статус Running і належить до того самого кластера. SQL-запити показали, що таблиця tablet_documents зберегла 17 документів, а раніше оновлений запис доступний через новий вузол. Це підтверджує коректну роботу кластера після вимкнення старого шарду та підключення нового.

Висновок.

У ході виконання лабораторної роботи було розгорнуто розподілений кластер YugabyteDB у середовищі WSL Ubuntu Linux із використанням Docker Compose. На початковому етапі було підготовлено робоче середовище, створено окрему директорію lab-4, перевірено наявність Docker, Docker Compose та допоміжних інструментів. Після цього було створено конфігураційний файл docker-compose.yml, у якому описано вузли кластера, спільну Docker мережу та окремі сховища даних для кожного вузла.

Під час розгортання кластера було запущено кілька вузлів YugabyteDB. Для кожного вузла перевірявся стан за допомогою команди yugabyted status. У результаті було підтверджено, що вузли мають статус Running, однаковий Universe UUID і працюють як частини одного розподіленого кластера. Через

вбудований веб інтерфейс YB Master було переглянуто загальний стан кластера, список tablet серверів, таблиці та параметри реплікації.

У практичній частині було створено власну базу даних lab4_shop, а також таблиці customers, orders, activity_logs і tablet_documents. Для виконання вимоги лабораторної роботи використовувалися власні тестові дані, відмінні від прикладів у методичних матеріалах. Було додано клієнтів, замовлення, події у форматі JSONB та окремі документи для перевірки розподілу по tablets.

Окремо було досліджено роботу tablets. Для цього таблицю tablet_documents було створено з поділом на кілька tablets, після чого було додано тестові документи і виконано підрахунок їх кількості за умовними діапазонами хешування. Це дозволило показати, що дані в YugabyteDB не зберігаються як один монолітний набір, а розподіляються між логічними частинами, які можуть розміщуватися на різних вузлах кластера.

Також було виконано оновлення одного документа і перевірено його читання з різних вузлів. Оновлений запис з doc_id = 2 мав однаковий вигляд при читанні через основний вузол, другий вузол і третій вузол. Це підтвердило, що кластер забезпечує узгоджений доступ до даних і дозволяє звертатися до них через різні вузли.

У наступному етапі кластер було розширено додатковими вузлами. Після запуску нових вузлів було дозаписано ще 5 документів, перевірено загальну кількість записів і виконано JOIN запит між кількома таблицями. Це показало, що YugabyteDB дозволяє працювати з розподіленими даними через звичайні SQL запити, приховуючи від користувача фізичне розміщення даних у кластері.

На завершальному етапі було виконано перевірку заміни шарду. Командою docker compose down було вимкнено попередній склад кластера, після чого через docker compose up було запущено новий склад із новим вузлом lab4-yb-node6 замість старого lab4-yb-node5. Після запуску було перевірено, що дані не були втрачені, таблиця tablet_documents містить 17 документів, а раніше оновлений документ доступний через новий вузол. Через Master UI було підтверджено, що кластер працює, новий вузол активний, а створені таблиці залишаються доступними.